SRS

Software Requirements Specification (SRS)

Document Meta

Project: Endur (Performance Review & Feedback Management System)
Team: 39_endur
Version: 1.0

---

1. Preface

1.1 Expected Readership

This document is intended for:

- **Project Developers (Team 39_endur):** To understand the technical specifications and constraints.
- **Domain Expert (Dean of Academics):** To validate that the operational workflows match university policy.
- **Project Evaluators:** To assess the depth of requirement analysis and system modeling.

1.2 Version History

Version	Date	Description	Author
1.0	2026-01-30	Initial Draft based on Dean's Interview	Team 39_endur

---

2. Introduction

2.1 Need for the System

The current end-semester feedback model fails on two critical fronts:

× Pain Point 1: Timing Latency

Feedback arrives too late to benefit the current batch and is often skewed by "Revenge Ratings" submitted after grades are released.

× Pain Point 2: Engagement Fatigue

The Dean reports that **only ~30% of students participate** because the process is viewed as a "chore" rather than a meaningful contribution.

2.2 System Functions

- **Continuous Feedback Cycles:** Allows multiple review windows per semester (Pulse Checks).
- **Role-Based Dashboards:** Distinct views for Student (Submission), Faculty (Reflection), and HOD (Oversight).
- **Gap Analysis:** Automated comparison between Faculty Self-Reflection and Student Perception.
- **Anonymity Preservation:** Cryptographic decoupling of student identity from feedback data.

2.3 Business & Strategic Objectives

✓ Goal: Accreditation

The system aligns with the institution's goal of obtaining **NBA/NAAC Accreditation** by generating auditable "Action Taken Reports" and ensuring a fair, data-driven faculty appraisal process.

---

3. Glossary

Refer to definitions in the repository for the complete Ubiquitous Language.

- **FeedbackCycle:** A defined, time-bound period for data collection.
- **GapAnalysis:** The quantitative difference between a professor's self-rating and the students' average rating.
- **AnonymizedReport:** A generated view where student identities are stripped to ensure psychological safety.
- **ActionReport:** A formal plan submitted by faculty to the HOD to address performance gaps.

4. User Requirements Definition

High-level services provided for the user, described in natural language.

4.1 Student Services

- **UR-01:** Students shall be able to submit **"Pulse Checks"** (Weekly/Monthly) which are lightweight (taking < 30 seconds) to ensure high participation.
- **UR-02:** Students shall be able to view a history of their past submissions for transparency.
- **UR-03 (Gamification):** The user interface shall utilize engaging elements (e.g., sliders, emojis, progress bars) rather than dense forms to combat "Survey Fatigue."

4.2 Faculty Services

- **UR-04:** Faculty shall be able to submit a **Self-Reflection** assessment before viewing student feedback.
- **UR-05:** Faculty shall be able to view performance trends across semesters without seeing individual student names.
- **UR-06:** Faculty shall be able to submit an **Action Report** if their performance scores trigger a system alert.

4.3 Administrative (HOD) Services

- **UR-07:** The Department Head (HOD) shall be able to view aggregated reports for all faculty members.
- **UR-08:** The HOD shall be able to identify "At-Risk" faculty based on consistent low-performance flags.

---

5. System Architecture

High-level overview of system modules.

The system follows a **Three-Tier Architecture**:

1. **Presentation Layer (Frontend):** React.js based Single Page Application (SPA) serving responsive forms and D3.js visualizations.
2. **Application Layer (Backend):** Node.js/Express REST API handling business logic (Validation, Aggregation, Auth).

3. **Data Layer (Database):** Relational Database (PostgreSQL/MySQL) storing Users, Feedback, and Course Mappings.

External Interfaces:

- **Auth Module:** Interacts with the University LDAP/Email service for identity verification.

---

6. System Requirements Specification

Detailed technical requirements and constraints.

6.1 Functional Requirements (FR)

FR-01 (Cycle Management)
The system shall strictly enforce submission windows; the "Submit" endpoint must reject requests received after `Cycle_End_Timestamp`.

FR-02 (Data Validation)
The system shall calculate the **PerformanceScore** using a weighted average if **Attendance Data** is available (Weight = 1.0 for >75% attendance).

FR-03 (Gap Logic)
The system shall compute `Gap = |Self_Score - Avg_Student_Score|` and highlight discrepancies > 2.0 points in red.

FR-04 (Locking Mechanism)
Once an **ActionReport** is submitted by a faculty member, it must become immutable (read-only).

FR-05 (Dual Modes)
The system must support two distinct feedback types:

1. **Type A (Pulse Check):** Weekly/Monthly, focuses strictly on "Content Delivery" and "Pace" (High frequency, low effort).
2. **Type B (Comprehensive):** End-Semester, covers all parameters including infrastructure and lab assistance.

FR-06 The "Heard" Loop (Engagement)
When a Faculty member marks an Action Item as "Completed" (e.g., "I slowed down the pace"), the system shall notify the relevant student batch (e.g., "Your feedback was heard!").

> **Rationale**

This closes the loop, proving to students that their feedback isn't a waste of time, directly solving the "30% participation" issue.

6.2 Non-Functional Requirements (NFR)

- **NFR-01 (Anonymity):** The `Student_ID` must **never** be retrievable from the `Feedback_Response` table.
- **NFR-02 (Scalability):** The system must support 500+ concurrent write operations within the final 10 minutes of a cycle.
- **NFR-03 (Availability):** The system must maintain 99.9% uptime during the active exam/feedback week.

6.3 Domain Requirements (DR)

Domain Constraint: Revenge Ratings

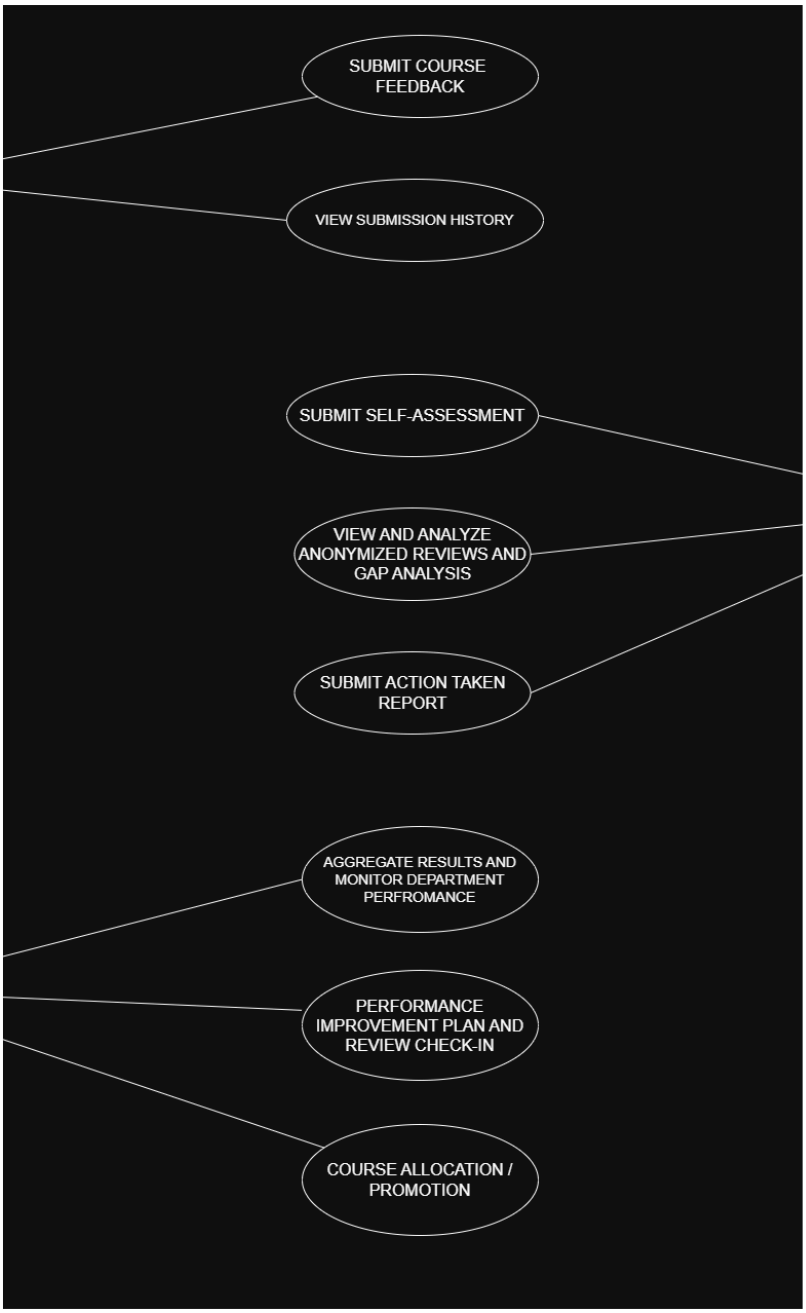
DR-01: The system must operationally lock all feedback cycles **before** the official release of semester grades to prevent grade-biased feedback.

---

7. System Models

7.1 Use Case Diagram

Visualizing Student, Faculty, and HOD interactions.



7.2 Activity Diagram

Detailing the "Gap Analysis" workflow.

(Placeholder: Insert Activity Diagram Mermaid Code Here)

7.3 Sequence Diagram

Detailing the "Secure Submission" API flow.

(Placeholder: Insert Sequence Diagram Mermaid Code Here)

---

8. System Evolution

Assumptions

- The university will provide an API for real-time attendance data. If not, attendance weighting will be manual in V1.0.

Future Scope

- Integration of AI-based **Sentiment Analysis** to summarize thousands of qualitative comments into a single "Department Sentiment Score."

---

9. Appendices

Appendix A: Technology Stack

- **Frontend:** React.js, Tailwind CSS, Recharts (for analytics).
- **Backend:** Node.js, Express.js, JWT (JSON Web Tokens).
- **Database:** PostgreSQL (for structured relational data).

Appendix B: Database Requirements

- **Tables Required:** `Users`, `Roles`, `Courses`, `FeedbackCycles`, `Submissions`, `ActionReports`.
- **Constraint:** `Submissions` table must have a composite unique key on `(Student_ID, Course_ID, Cycle_ID)` to prevent duplicate feedback.